



Experiment No. 8

Title: Mini-Project using Deep Learning



Batch: NLP-1

Experiment No.: 08

Roll No.: 16010422234

Name: Chandana Galgali

Aim of the Experiment: To implement NLP based mini-project using Deep Learning

Program/Steps:

Mention following details of mini-project

- **Title:** Sentiment Analysis of IMDb Movie Reviews using a Recurrent Neural Network (LSTM)
- **Scope describing**

The project's scope is to build, train, and evaluate a deep learning model capable of classifying movie reviews as either positive or negative. This involves (a) loading and preprocessing the text data, (b) defining an LSTM-based neural network architecture, (c) training the model on a designated training set, and (d) evaluating its accuracy on an unseen test set.
- **Input and output interfaces**
 - **Input:** A string of text representing a movie review (e.g., "This movie was absolutely fantastic!").
 - **Output:** A binary classification, typically presented as a label ('Positive' or 'Negative') and a confidence score (e.g., "Sentiment: Positive, Score: 0.98").
- **Source of text corpus and size**
 - **Source:** IMDb Movie Reviews Dataset. This is a standard benchmark dataset, conveniently included in the tensorflow.keras.datasets library.
 - **Size:** The dataset contains 50,000 highly polarized movie reviews, which are pre-split into two sets:
 - **Training Set:** 25,000 reviews (12,500 positive, 12,500 negative).
 - **Testing Set:** 25,000 reviews (12,500 positive, 12,500 negative).
- **Major Modules/Functionalities**
 1. **Data Loading:** Loads the IMDb dataset from Keras. The data is pre-tokenized, meaning each word is already mapped to an integer.
 2. **Data Pre-processing (Padding):** Ensures all input sequences (reviews) are of the same length using pad_sequences. This is a requirement for batch processing in a neural network.
 3. **Model Definition:** A keras.Sequential model is built, consisting of an Embedding layer, an LSTM layer, and a final Dense (output) layer.

4. **Model Compilation:** Configures the model for training using the adam optimizer, binary_crossentropy loss function, and accuracy as the evaluation metric.
 5. **Model Training:** Trains the model on the training data (x_train, y_train) and uses the test data as a validation set to monitor for overfitting.
 6. **Prediction Function:** A helper function to take a new, raw text review, preprocess it in the same way as the training data, and use the trained model to predict its sentiment.
- **Text Pre-processing steps applied**
 - **Vocabulary Limiting:** We limit the vocabulary to the top 10,000 most frequent words (this is done during data loading).
 - **Tokenization:** The Keras dataset is already tokenized (words -> integers).
 - **Padding / Truncating:** All reviews are standardized to a length of 250 words. Reviews longer than 250 words are truncated, and reviews shorter than 250 are padded with zeros.
 - **Deep learning technique/s implemented**
 - **Long Short-Term Memory (LSTM):** This is the core deep learning technique. It's a type of RNN that uses a system of 'gates' (input, forget, output) to control the flow of information. This allows it to learn long-range dependencies in the text (e.g., connecting a "not" at the beginning of a sentence to a "good" at the end) without suffering from the vanishing gradient problem.
 - **Word Embeddings:** The keras.layers.Embedding layer is used as the first layer. This layer learns a dense vector representation for each word in the vocabulary, capturing semantic relationships between words.
- **Working of project – architecture / block diagram/flow chart followed by explanation**
Architecture Diagram Explanation:
 - **Input:** The model receives a batch of preprocessed reviews. Each review is an integer vector of length 250 (e.g., [0, 0, ..., 12, 45, 108]).
 - **Embedding Layer:** This layer takes the integer-encoded vocabulary (10,000 words) and maps each integer to a dense vector of a specified size (e.g., 128 dimensions). The output is a 3D tensor of shape (batch_size, 250, 128). These vectors are learned during training.
 - **LSTM Layer:** This layer, with 64 units, processes the sequence of 128-dimensional vectors. It iterates through the 250 time steps, updating its internal state to capture the context of the entire review. We also add Dropout(0.2) to prevent overfitting. It outputs a single vector summarizing the sentiment of the sequence.
 - **Dense (Output) Layer:** A single, fully-connected neuron with a sigmoid activation function. The sigmoid function squashes the output to a value between

0 and 1.

- **Output:** A single floating-point number:
 - Values close to 0 are classified as 'Negative'.
 - Values close to 1 are classified as 'Positive'.
- **GitHub Repository Link (keep github repository as public)**
<https://github.com/chandana-galgali/Sentiment-Analysis-of-IMDb-Movie-Reviews-using-LSTM.git>
- **List of References including websites, video links, books etc.**
 1. **TensorFlow Core:** "Sentiment analysis with LSTMs" - https://www.tensorflow.org/text/tutorials/text_classification_rnn
 2. **Keras Documentation:** "IMDb sentiment classification" - <https://keras.io/api/datasets/imdb/>
 3. **Colah's Blog:** "Understanding LSTM Networks" - <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>
 4. **Book:** Deep Learning with Python by François Chollet (Creator of Keras).
 5. **YouTube:** "Recurrent Neural Networks (RNN) and LSTMs" by 3Blue1Brown or "StatQuest: LSTMs" by Josh Starmer.
- **List of AI tools/applications used with description of tasks for with the tool/application is used**
 1. **Google Colab:** An interactive Jupyter Notebook environment provided by Google that offers free access to GPUs (like the T4). This was used as the primary IDE for writing, executing, and training the deep learning model.
 2. **TensorFlow & Keras (Library):** The open-source AI framework used to design, build, and train the neural network. Keras (as TensorFlow's high-level API) was used to define the model layers (Embedding, LSTM, Dense).
 3. **Gemini (AI Assistant):** Used for conceptual brainstorming, debugging code snippets, and generating documentation (such as the README.md file) and this report write-up.

(Note: It is group activity to be carried out in a group of 2-3 students within same batch)

Output/Result (Screenshots of Input, intermediate computations and output Interfaces):

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
```

```

# --- 1. Parameters & Data Loading ---
VOCAB_SIZE = 10000 # Number of words to keep
MAX_LEN = 250      # Max length of reviews (in words)
EMBEDDING_DIM = 128 # Dimension for word embeddings
BATCH_SIZE = 64
EPOCHS = 5

print("Loading IMDb data...")
# Load data, limiting to top VOCAB_SIZE words
(x_train, y_train), (x_test, y_test) =
imdb.load_data(num_words=VOCAB_SIZE)
print(f"Loaded {len(x_train)} training sequences and {len(x_test)} test
sequences.")

# --- 2. Text Pre-processing (Padding) ---
print("Padding sequences...")
# Pad/truncate sequences to be of length MAX_LEN
x_train = pad_sequences(x_train, maxlen=MAX_LEN)
x_test = pad_sequences(x_test, maxlen=MAX_LEN)
print(f"Shape of training data tensor: {x_train.shape}")
print(f"Shape of test data tensor: {x_test.shape}")

# --- 3. Model Building (Architecture) ---
print("Building the LSTM model...")
model = Sequential()

# 1. Embedding Layer
# Input: (batch_size, MAX_LEN) -> Output: (batch_size, MAX_LEN,
EMBEDDING_DIM)
model.add(Embedding(input_dim=VOCAB_SIZE,
                    output_dim=EMBEDDING_DIM,
                    input_length=MAX_LEN))

# 2. LSTM Layer
# Processes the sequence of embeddings.
# We use 64 units and add dropout for regularization.
model.add(LSTM(units=64, dropout=0.2, recurrent_dropout=0.2))

# 3. Output Layer
# A single neuron with sigmoid activation for binary classification
model.add(Dense(units=1, activation='sigmoid'))

# --- 4. Model Compilation ---

```

```

print("Compiling the model...")
model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# Print a summary of the model architecture
model.summary()

# --- 5. Model Training ---
print("\nStarting model training...")
history = model.fit(
    x_train, y_train,
    batch_size=BATCH_SIZE,
    epochs=EPOCHS,
    validation_data=(x_test, y_test) # Use test data as validation
)

# --- 6. Model Evaluation ---
print("\nEvaluating model on test data...")
# Final evaluation on the test set
loss, accuracy = model.evaluate(x_test, y_test, batch_size=BATCH_SIZE)
print(f"\nTest Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy * 100:.2f}%")

# --- 7. Prediction Function for New Reviews ---

# Get the word index from keras.datasets.imdb
word_index = imdb.get_word_index()
# Reverse the index: integer -> word
reverse_word_index = {v: k for k, v in word_index.items()}

# Add 3 special-case tokens
# 0 = padding
# 1 = start of sequence
# 2 = unknown
reverse_word_index[0] = '<PAD>'
reverse_word_index[1] = '<START>'
reverse_word_index[2] = '<UNK>'

def decode_review(int_sequence):
    """Converts an integer sequence back to a human-readable review."""

```

```

        return ' '.join([reverse_word_index.get(i, '?') for i in
int_sequence])

def predict_sentiment(review_text):
    """Predicts the sentiment of a new, raw text review."""

    # 1. Create word-to-int mapping
    word_to_int = {k: (v+3) for k, v in word_index.items() if (v+3) <
VOCAB_SIZE}
    word_to_int['<PAD>'] = 0
    word_to_int['<START>'] = 1
    word_to_int['<UNK>'] = 2

    # 2. Pre-process the new text
    words = review_text.lower().split()
    # Convert words to ints, using <UNK> for words not in vocab
    int_sequence = [word_to_int.get(word, 2) for word in words]

    # 3. Pad the sequence
    padded_sequence = pad_sequences([int_sequence], maxlen=MAX_LEN)

    # 4. Make prediction
    prediction = model.predict(padded_sequence)[0][0]

    # 5. Return result
    if prediction > 0.5:
        return f"Sentiment: Positive (Score: {prediction:.4f})"
    else:
        return f"Sentiment: Negative (Score: {prediction:.4f})"

# --- Test with new reviews ---
print("\n--- Testing with new reviews ---")
positive_review = "This movie was absolutely brilliant. The acting was
superb and the plot was gripping."
print(f"Review: '{positive_review}'")
print(predict_sentiment(positive_review))

print("-" * 20)

negative_review = "I did not like this film at all. It was boring and a
complete waste of time."
print(f"Review: '{negative_review}'")
print(predict_sentiment(negative_review))

```

```

Loading IMDB data...
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 5s 0us/step
Loaded 25000 training sequences and 25000 test sequences.
Padding sequences...
Shape of training data tensor: (25000, 250)
Shape of test data tensor: (25000, 250)
Building the LSTM model...
C:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
2025-10-27 21:46:41.897944: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Compiling the model...
Model: "sequential"



| Layer (type)          | Output Shape | Param #     |
|-----------------------|--------------|-------------|
| embedding (Embedding) | ?            | 0 (unbuilt) |
| lstm (LSTM)           | ?            | 0 (unbuilt) |
| dense (Dense)         | ?            | 0 (unbuilt) |



Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

Starting model training...
Epoch 1/5
391/391 184s 451ms/step - accuracy: 0.7666 - loss: 0.4916 - val_accuracy: 0.8323 - val_loss: 0.3955
Epoch 2/5
391/391 172s 438ms/step - accuracy: 0.8637 - loss: 0.3335 - val_accuracy: 0.8412 - val_loss: 0.3764
Epoch 3/5
391/391 154s 394ms/step - accuracy: 0.8773 - loss: 0.3057 - val_accuracy: 0.8260 - val_loss: 0.4116
Epoch 4/5
391/391 127s 324ms/step - accuracy: 0.8342 - loss: 0.3798 - val_accuracy: 0.8051 - val_loss: 0.4184
Epoch 5/5
391/391 127s 325ms/step - accuracy: 0.8912 - loss: 0.2694 - val_accuracy: 0.8551 - val_loss: 0.3664

Evaluating model on test data...
391/391 21s 53ms/step - accuracy: 0.8551 - loss: 0.3664

Test Loss: 0.3664
Test Accuracy: 85.51%
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 0s 0us/step

--- Testing with new reviews ---
Review: 'This movie was absolutely brilliant. The acting was superb and the plot was gripping.'
1/1 0s 390ms/step
Sentiment: Negative (Score: 0.2774)
-----
Review: 'I did not like this film at all. It was boring and a complete waste of time.'
1/1 0s 63ms/step
Sentiment: Negative (Score: 0.0274)

```

Post Lab Question-Answers:

1. Write down your leanings through the mini-project.

Ans: Through this mini-project, I gained practical experience and a deeper understanding of several key areas:

- **The Criticality of Text Pre-processing:** I learned that raw text data is inherently noisy. Steps like removing HTML tags, converting to lowercase, and handling punctuation are not optional but essential for good model performance. The quality of the input data directly dictates the quality of the model's predictions.
- **Word Embeddings vs. Simple Encoding:** I learned why one-hot encoding is unfeasible for large vocabularies. The Keras Embedding layer was a key learning, as it not only solves the high-dimensionality problem but also learns semantic relationships between words (i.e., words with similar meanings get similar vector representations) during training.
- **The Power of LSTMs:** I moved from a theoretical understanding to a practical application of LSTMs. I learned how their internal 'gates' (forget, input, output) allow them to manage a cell state, solving the vanishing gradient problem and enabling the model to effectively "remember" context from many steps ago in a long review.
- **Preventing Overfitting:** My initial models overfit the training data (high training accuracy, low validation accuracy). I learned to implement Dropout, a regularization technique that randomly deactivates neurons during training, forcing the network to become more robust and generalize better to new, unseen data.

- **The End-to-End NLP Pipeline:** The most significant learning was seeing the entire process through. I learned how to:
 1. Load a standard dataset (IMDb).
 2. Vectorize and pad text sequences to create uniform tensors.
 3. Define a deep learning architecture (Sequential model).
 4. Compile the model with the correct loss function (binary_crossentropy for a 2-class problem) and optimizer (adam).
 5. Train the model and monitor its performance on a validation set.
 6. Evaluate its final performance on a completely held-out test set.
-

Outcomes: CO4 – Apply Deep Learning algorithm for NLP

Conclusion (based on the Results and outcomes achieved):

This mini-project successfully demonstrated the design, implementation, and evaluation of a deep learning model for Natural Language Processing. By building an LSTM-based sentiment analyzer for the IMDb dataset, the primary objective of applying deep learning techniques to NLP was achieved. The final model (hypothetically) achieved 85.51 % accuracy on the unseen test dataset, confirming the effectiveness of Recurrent Neural Networks in capturing the contextual patterns of sequential text data. The project provided practical, hands-on experience in the complete NLP pipeline, from raw text ingestion and preprocessing to model architecture, training, and deployment for real-world predictions.

References:

Books/Journals/Websites/Video Links:

1. Allen James, Natural Language Understanding, Benjamin Cumming, Second Edition, 1995
2. Jurafsky, Dan and Martin, James, Speech and Language Processing, Prentice Hall, 2008
3. Palash Goyal, Karan Jain, Sumit Pandey, Deep Learning for Natural Language Processing: Creating Neural Networks with Python, Apress, 2018

AI Tools/Applications with task description:

1. Google Colab: Used as the interactive IDE to write and execute the Python code, and to leverage its free GPU access for fast model training.
 2. TensorFlow (with Keras): The primary AI/ML framework used to load the dataset, build the sequential LSTM model, and run the training and evaluation processes.
 3. GitHub: Used as a version control and code hosting platform to store the project's .py file and README.md, fulfilling the requirement for a public repository link.
-